# 45. Linux for AI Infrastructure

## 45. Linux for AI Infrastructure

**Phase:** Phase 11 — Containers and cloud Linux
**Estimated time:** 2 days
**Slug:** `45-ai-infra-linux`

### Learning objectives

- Prepare GPU hosts at OS level (drivers/toolkit conceptually)
- Tune file limits and disk for datasets
- Isolate model servers with systemd/cgroups
- Observe GPU and host metrics

### Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

### Why this matters

AI serving looks exotic, but at 3am it is ordinary Linux trouble: a model server that dies with “Too many open files”, a loader that thrashes because the dataset sits on a slow disk, a GPU process that gets OOM-killed because a neighbor leaked host memory. The people who keep inference reliable know limits, cgroups, systemd, and basic observability. GPU drivers and CUDA are important, but they sit on top of the same kernel features you already study.

One knob bites first and most often: the open-file-descriptor limit, `nofile`. A busy inference server holds sockets for every client, plus model shards, tokenizer files, logs, and worker pipes. The default soft limit of 1024 is far too low, so the process fails under load, not at startup.

## Core concepts

| Area | What to know | Where to look |
|---|---|---|
| File descriptors | per-process cap (nofile) | `ulimit -n`, `LimitNOFILE=` |
| Memory | RAM, page cache, swap, cgroup limit | `free -h`, `MemoryMax=` |
| Disk | dataset and model weights read speed | `iostat -x 1`, `df -h` |
| GPU (host side) | driver, device nodes, utilization | `nvidia-smi`, `/dev/nvidia*` |
| Processes | one model server, N workers | `systemctl status`, `ps` |
| Isolation | cgroups, user, namespaces | `systemd-cgls`, `systemd-run` |

```
clients --> proxy --> model server (systemd unit)
                        |  LimitNOFILE=65536   MemoryMax=48G
                        |  User=model          TasksMax=...
                        v
        page cache <-- weights/datasets on NVMe
                        |
                    GPU driver --> /dev/nvidia0 (utilization, memory)
  watch: nofile, RSS, page cache, disk wait, GPU mem, 5xx rate
```

## Worked examples

Example 1: see the limits a running server really has.

```
pid=$(pgrep -f model-server | head -1)
grep -E 'open files|processes' /proc/$pid/limits
ls /proc/$pid/fd | wc -l
```

Compare the count with the soft limit. If they are close, you are about to fail.

Example 2: raise the limit for a systemd service, the correct way.

```
[Service]
User=model
LimitNOFILE=65536
MemoryMax=48G
TasksMax=4096
ExecStart=/opt/model/bin/server --port 8000
```

```
sudo systemctl daemon-reload
sudo systemctl restart model.service
systemctl show model.service -p LimitNOFILE
```

Editing `/etc/security/limits.conf` does not affect systemd services; use the unit file or a drop-in.

Example 3: confirm dataset disk is not the bottleneck.

```
df -h /data
iostat -x 1 3
```

High `%util` with large `await` while the GPU idles means storage is feeding the model too slowly.

Example 4: host-side GPU sanity.

```
nvidia-smi
ls -l /dev/nvidia*
journalctl -k | grep -i -E 'nvrm|xid' | tail
```

An Xid error in the kernel log points at driver or hardware, not your app.

Example 5: was it the OOM killer?

```
journalctl -k | grep -i 'out of memory'
systemctl status model.service | head
```

## Mini-lab

1. In a shell run `ulimit -n`, then `ulimit -Hn`. Note the soft and hard values.
2. Write a tiny script that opens many files and watch it fail at the soft limit: `python3 -c "f=[open('/dev/null') for _ in range(5000)]"`.
3. Run it again after `ulimit -n 10000` in the same shell.
4. Create a unit for a dummy server (`python3 -m http.server 8000`) with `LimitNOFILE=4096` and `MemoryMax=200M`. Start it.
5. Read `/proc/PID/limits` for its main process and confirm 4096.
6. Write the three metrics you would alert on for a model host.

## Decision guide / failure modes

| Symptom | Likely cause | First check |
|---|---|---|
| "Too many open files" | low nofile | `/proc/PID/limits`, `LimitNOFILE` |
| Killed with no log | cgroup or global OOM | `journalctl -k`, `MemoryMax` |
| Slow start, idle GPU | slow disk or cold cache | `iostat -x`, warm the cache |
| GPU not found | driver mismatch, missing nodes | `nvidia-smi`, `dmesg` |
| Latency spikes | swap, CPU throttle, noisy neighbor | `vmstat 1`, `cpu.stat` |

Failure modes: raising `limits.conf` and expecting systemd to obey; running the model server as root; unlimited memory so one leak takes down the host; and no graceful drain, so deploys drop in-flight requests. Always measure the real limit of the live process, not the one you think you set.

## Common mistakes

- Sharing one GPU host with zero isolation
- Datasets on tiny root disks
- No monitoring of GPU temp/utilization

## Exercises

Work in `exercises/45-ai-infra-linux/`. Read that folder's `README.md` fully before coding.

```
cd exercises/45-ai-infra-linux
chmod +x test.sh exercise.sh
./test.sh
```

### What you will build

Open `exercise.sh` and implement `inference_limit_knob`. It prints the name of the one resource-limit knob, written exactly as the lowercase `ulimit` resource name, that a busy inference server most commonly exhausts first because it holds many sockets and files. Print only that word. The test does not touch the system. After it passes, perform mini-lab steps 1 to 5 and keep the output of `grep 'open files' /proc/PID/limits` in your notes. Then write the systemd directive that raises the same limit for a service and say why `limits.conf` alone does not change it for services.

### Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

**Do not open `solutions/45-ai-infra-linux/` until you have an honest failing attempt.**

## Self-check

Try to answer without scrolling up:

1. Why nofile for inference servers?
2. Where put large datasets?
3. Why cgroups/systemd isolation?

## Further reading

- https://docs.nvidia.com/datacenter/cloud-native/